NEST.PH

Technical Walkthrough

How our system works, explained simply, with the code behind each feature

Defense preparation guide · Built for Pureza Station Dormitory
October 2026

Contents

Contents 2
1. How to use this guide 4
2. The big picture 5
2.1 What NEST.PH is 5
2.2 Who uses it 5
2.3 The tools we built it with 5
2.4 Five words you will hear a lot 6
2.5 Where things live in the project 6
3. The whole journey in one page 8
4. Logging in and account security 9
What it does 9
How it works 9
5. Rooms, beds and VR tours 11
What it does 11
How it works 11
6. Applying and signing the documents online 12
What it does 12
How it works 12
7. Approving an application 13
What it does 13
How it works (approve) 13
8. Move-in payment 14
What it does 14
How it works 14
9. Monthly billing 15
What it does 15
How it works 15
Proration: mid-month move-in and move-out 15
10. Paying bills: proof upload, approval, cash 18
What it does 18
How it works 18
11. Grace period and late penalty 19
What it does 19

How it works 19
12. Late-payment follow-up (6-stage delinquency escalation) 20
What it does 20
How it works 20
13. Maintenance tickets and automatic priority 22
What it does 22
How it works (rule-based, in English and Filipino) 22
14. Reviews and automatic moderation 23
What it does 23
What it checks 23
15. Notifications, SMS and email 24
16. Reports and forecasting 25
What it does 25
How the forecast works (simple averages, no AI) 25
17. Tenant records, move-out and dorm settings 26
Tenant Manager 26
Moved-out tenants 26
Dormitory Profile (settings) 26
Activity Log 26
18. Everything that runs automatically every day 27
19. Security and data protection, in short 28
20. Limitations and honest answers 29
21. Quick-answer sheet 30

(If the list above is empty, right-click it in Word and choose "Update Field".)

1. How to use this guide

This guide explains what NEST.PH does and how each part works behind the screen. Every feature section has the same four parts:

- **What it does**: the feature in one or two sentences.
- **How it works**: the steps the system follows, in order.
- **Where the code is**: the exact file to open if a panelist says "show us the code."
- **Questions the panel might ask**: with suggested answers.

Tip for the defense: when you are asked to show code, open the file in VS Code and press **Ctrl+P**, type the file name (for example EscalationService), then press **Ctrl+Shift+O** to jump to a function by name (for example processAll).

2. The big picture

2.1 What NEST.PH is

NEST.PH is a website that runs a dormitory from start to finish. A person can find a room online, take a virtual (VR) tour, apply, sign the contract, pay the move-in fee, pay monthly rent, report problems, and finally move out, all in one system. The owner and staff use the admin side to manage rooms, check payments, send bills, follow up on late payers, and print reports.

Our real client is **Pureza Station Dormitory**. Their signed documents (Tenant Agreement, Rules and Regulations, Payments and Fees Schedule) are the source of the business rules: due date on the 1st, 3-day grace period, one-time 10% late penalty, 3-month minimum stay, one month deposit plus one month advance.

2.2 Who uses it

User	What they do	Where they log in
Visitor (not logged in)	Browse rooms and beds, VR tours, dorm info and rules, send an inquiry, apply.	No login
Tenant	Pay move-in fee and rent, upload proof of payment, see bills, file tickets, read announcements, write a review.	/login/tenant
Admin (owner/staff)	Manage rooms and beds, approve applications and payments, generate bills, handle late payers, tickets, reports, settings.	/login/admin

2.3 The tools we built it with

Tool	What it is, simply	What we use it for
PHP + Laravel 12	The programming language and a ready-made "toolkit" (framework) for websites.	All the logic: logins, billing, payments, rules.
Blade	Laravel's way of writing web pages with fill-in-the-blanks.	Every page you see (resources/views).
Tailwind CSS + JavaScript	Styling and small in-page behavior.	How pages look; buttons, pop-ups, forms.
MySQL database	An organized set of tables, like very strict spreadsheets.	Stores tenants, rooms, beds, bills, payments, tickets…
DomPDF	Turns a web page into a PDF.	Contracts, receipts, demand letters, eviction notices, reports.
PhpSpreadsheet	Creates Excel files.	Exporting reports to .xlsx.

Tool	What it is, simply	What we use it for
Textbee	An SMS gateway (an Android phone that sends texts for us).	Payment reminders and late-payment texts.
Email (Laravel Mail)	Sends emails.	Login details, password-reset codes, move-in permit, notices.
Pannellum (VR viewer)	Shows 360° photos you can drag around.	Virtual room tours.

2.4 Five words you will hear a lot

In plain words: A website request goes like this: **Route** (the address) → **Middleware** (the guard at the door) → **Controller** (the worker who does the job) → **Model** (talks to the database table) → **View** (the page sent back).

Word	Meaning	Example in our code
Route	A web address and which code handles it.	routes/web.php: /payments/{payment}/approve → PaymentController@approveProof
Middleware	A check that runs before the page opens.	RestrictDelinquentTenant blocks a late payer from everything except Billing.
Controller	The code that does the job for a page or button.	ApplicationController@approve creates the tenant and contract.
Model	A PHP class that stands for one database table.	BillingStatement = the billing_statements table.
Service	A helper class for a big job used in many places.	EscalationService runs the 6-stage late-payment process.
Scheduler / command	Code that runs automatically every day.	routes/console.php runs billing and escalation daily.
Transaction	A group of database changes that all succeed or all get cancelled.	Approving an application: tenant + contract + bill are saved together.

2.5 Where things live in the project

Folder	What is inside
routes/web.php	Every web address in the system, grouped into Public, Auth, Admin and Tenant.
routes/console.php	The daily automatic jobs (billing, escalation, overdue applications, reservation expiry).

Folder	What is inside
app/Http/Controllers	The controllers (the "workers"). The Api folder holds the main business ones.
app/Http/Middleware	The "guards" (admin only, tenant only, move-in check, delinquency check, moved-out check).
app/Models	One file per database table (Tenant, Room, Bed, BillingStatement, Payment, Penalty…).
app/Services	Big reusable logic (EscalationService, TicketPriorityClassifier, ReviewModerationService, TextbeeService…).
app/Console/Commands	Commands you can run by hand or on a schedule (escalation:process, reservations:expire…).
resources/views	All the pages (Blade files), including documents/ for the contract PDFs and pdfs/ for letters.
database/migrations	The "blueprints" that create each database table.
database/seeders	Sample data. PurezaStationSeeder loads the real dorm settings; DemoDataSeeder adds dummy tenants.

3. The whole journey in one page

This is the life of one tenant in the system. Each step links to a section later in this guide.

1. Visitor browses rooms and beds and takes a VR tour (Section 5).
2. Visitor fills in the application form and signs the three documents on screen. The chosen bed becomes **Reserved** (Section 6).
3. Admin approves. The system creates the tenant account, the lease contract and a **move-in bill** (1 month deposit + 1 month advance) and emails the login details (Section 7).
4. Tenant logs in, is kept on the Move-In pages until the move-in fee is paid, uploads proof of payment (Section 8).
5. Admin approves the proof. The bed becomes **Occupied** and the tenant becomes **Active** (Section 8).
6. Every month the system creates a bill, due on the 1st (Section 9).
7. Tenant pays and uploads proof; admin approves or rejects (Section 10).
8. If not paid by the end of the 3-day grace period, a one-time late penalty is added (Section 11) and the 6-stage follow-up starts (Section 12).
9. Along the way: tickets for repairs (Section 13), announcements, notifications (Section 15).
10. At the end: move-out, deposit refund, and a review that is checked automatically (Sections 14 and 17).

4. Logging in and account security

What it does

There are two separate login pages, one for tenants and one for admins. Each page only accepts its own kind of account. Too many wrong passwords locks the account for a while. Forgotten passwords are reset with a 5-digit code sent by email.

How it works

1. The person types email and password. Laravel checks the password against the **hashed** (scrambled, cannot be reversed) version in the database.
2. If wrong, the system counts the failure. **5 wrong tries within 5 minutes locks login for 15 minutes.**
3. If correct, the system checks the person used the right page. An admin on the tenant page (or the opposite) is logged out with a message.
4. If the account was deactivated, login is refused. One exception: a tenant who already moved out may log in only to leave a review.
5. For admins, the login is recorded in the Login Tracker (time, device).

Where the code is: `app/Http/Controllers/Api/AuthController.php (login)`

Lockout rule (shortened)

```
private const MAX_ATTEMPTS = 5;
private const ATTEMPT_WINDOW_SECONDS = 300; // 5 minutes
private const LOCKOUT_SECONDS = 900;        // 15 minutes

if (! Auth::attempt($credentials, $remember)) {
    RateLimiter::hit($attemptsKey, self::ATTEMPT_WINDOW_SECONDS);
    if (RateLimiter::attempts($attemptsKey) >= self::MAX_ATTEMPTS) {
        RateLimiter::hit($lockKey, self::LOCKOUT_SECONDS);   // lock
        ...
```

Password reset

1. User enters email. If it exists, the system makes a random 5-digit code, saves only its **hash**, and emails the code.
2. The reply is the same whether the email exists or not ("If that email is registered…") so strangers cannot use it to find out who has an account.
3. User enters the code. It is valid for **10 minutes**.
4. On the final step the code is checked **again** (we never trust that the earlier step happened), then the new password is saved, hashed.

Where the code is: `app/Http/Controllers/Auth/PasswordResetCodeController.php (send, verify, reset, codeIsValid)`

Admin privileges

Not every admin can manage other admins. Only an admin with the **manage_users** privilege can open the Admin Privileges page, add an admin, change privileges or revoke access. This is enforced by a middleware, so even typing the address directly is blocked.

Where the code is: `app/Http/Middleware/EnsureCanManagePrivileges.php`, `app/Http/Controllers/AdminPrivilegeController.php`

Questions the panel might ask

Q: How are passwords stored?

A: Never as plain text. Laravel stores a hash (bcrypt). Even we cannot read a user's password. Reset codes are hashed too.

Q: What stops someone from guessing passwords?

A: Rate limiting: after 5 wrong attempts within 5 minutes, login is locked for 15 minutes. See AuthController.php, the constants at the top.

Q: How do you stop a tenant from opening admin pages?

A: Every admin address is inside a route group with the "admin" middleware (EnsureUserIsAdmin). If the user's role is not admin, the request is refused before any admin code runs.

Q: How does a restricted user get blocked even if they type the URL?

A: The checks are on the server (middleware), not just hidden buttons. The browser never decides access.

5. Rooms, beds and VR tours

What it does

The admin sets up floors, rooms and beds. Vacancy is tracked **per bed**, not just per room, because a 4-bed room can be half full. Visitors see which beds are free. Each room can have photos and a 360° VR tour with clickable hotspots to move between scenes.

How it works

- A bed has one of four statuses: **vacant, reserved, occupied, maintenance**.
- A room's own status (available / full / maintenance) is recalculated from its beds every time a bed changes (Room::syncStatusFromBeds).
- Room prices come from **room types** set in Dormitory Profile. The per-bed rate is the room rate divided by number of beds (Room::perBedRate).
- Bed status changes automatically: Vacant → Reserved (application sent) → Occupied (move-in fee paid). Rejected or expired applications put it back to Vacant.
- VR: admin uploads a panorama photo for each scene and clicks on the photo to place hotspots ("go to bathroom"). Phone panoramas often lack the ceiling and floor, so PanoramaFillService "paints" a soft top and bottom from the photo's edge colors so the viewer never shows black. The original photo is never changed.

Where the code is: `app/Http/Controllers/VacancyController.php, app/Models/Room.php, app/Http/Controllers/VrTourController.php, app/Services/PanoramaFillService.php`

Questions the panel might ask

Q: Why track beds instead of rooms?

A: Dorms rent by bedspace. A room with 3 of 4 beds taken is still available, so the visitor must see the free bed.

Q: Can two people apply for the same bed?

A: No. When an application is sent, the system checks the bed is still vacant and that no other pending application exists for it, then marks it Reserved.

Q: How did you make the VR tour?

A: We use the Pannellum 360° viewer in the browser. Scenes and hotspots are saved in the vr_scenes and vr_hotspots tables, so the admin can build tours without code.

6. Applying and signing the documents online

What it does

A visitor picks a bed, fills in personal and emergency-contact details, uploads IDs, reads Pureza's three documents (Agreement, Rules, Payments and Fees Schedule) and signs them on screen with a finger or mouse. The emergency contact signs on a separate pad.

How it works

1. The form is checked (validation). Examples: applicant must be **at least 18**; start date cannot be in the past; the end date must respect the **minimum stay (3 months)**; there must be a contact number or an email; the emergency contact's ID is required; files must be JPG, PNG or PDF up to 5 MB; data-privacy consent must be ticked.
2. When the applicant signs, the system builds a PDF of the three documents with their details and both signatures filled into the blanks, and saves it.
3. It also saves a **fingerprint** of the details that were signed. If the applicant changes anything after signing (for example the bed or name), the fingerprint no longer matches and they must sign again. This guarantees the signed PDF matches the application.
4. The bed is checked again (still vacant, no other pending application), then the application is saved and the bed becomes **Reserved**.

Where the code is: `app/Http/Controllers/Api/ApplicationController.php (store, signContract), app/Services/TenancyDocuments.php, resources/views/documents/`

A few of the application checks

```
'birthdate' => ['required', 'date', 'before_or_equal:' . now()->subYears(18)->toDateString()],
'emergency_contact_id' => ['required', 'file', 'mimes:jpg,jpeg,png,pdf', 'max:5120'],
'dpa_consent' => ['required', 'accepted'],
...
if ($bed->status !== 'vacant') {
    return response()->json(['message' => 'That bedspace is no longer available...'], 409);
}
```

Questions the panel might ask

Q: Is the e-signature legally valid?

A: It is a drawn image signature placed on a PDF together with the date, plus a saved record (fingerprint) of exactly what was signed. Under the E-Commerce Act (RA 8792) electronic signatures are recognized; the owner can also upload their own PDF version of each document if they prefer.

Q: What if someone edits the form after signing?

A: The fingerprint check fails and they are asked to sign again, so the signed copy always matches the submitted details.

Q: How do you handle Data Privacy?

A: The applicant must tick the data privacy consent before submitting. The emergency contact only gets billing reminders if they separately agree (Agreement 9.3).

7. Approving an application

What it does

The admin reviews an application and approves it, rejects it (with a reason), or asks the person to re-apply. Approving creates everything the new tenant needs in one go.

How it works (approve)

1. Checks: only a **pending** application can be approved, and its bed must still be **Reserved** for it (prevents giving one bed to two people).
2. Looks for a **returning tenant** (same email, or same phone number and last name). If found, the old account is reused; the admin may give a discount.
3. Inside one **transaction** (all-or-nothing): create the tenant and their login with a temporary password; create the **lease contract** (start date, end date, monthly rate minus any discount); create the **move-in bill = monthly rate × 2** (one month advance + one month deposit, kept as separate amounts).
4. If no end date was given, the contract ends on the last day of the 3rd month (minimum stay, Agreement 2.2).
5. Because the applicant already signed online, the contract is immediately **Active**.
6. After saving, the login details are emailed. If the email fails, the approval is not undone.
7. The bed stays **Reserved**, not Occupied, because the tenant has not paid yet.

Reject: a reason is required, the bed goes back to **Vacant**, and the applicant is emailed the reason.

Where the code is: `app/Http/Controllers/Api/ApplicationController.php (approve, reject, requestReapplication, findReturningTenant)`

Creating the move-in bill

```
$moveInFeeAmount = $monthlyRate * 2; // 1 month deposit + 1 month advance

BillingStatement::create([
    'type' => 'move_in',
    'base_rent' => $moveInFeeAmount,
    'advance_amount' => $monthlyRate,
    'deposit_amount' => $monthlyRate,
    'total_amount' => $moveInFeeAmount,
    'status' => 'unpaid',
]);
```

Questions the panel might ask

Q: Why use a transaction?

A: So we never end up with half the data, like a tenant with no contract. If any step fails, all of it is cancelled.

Q: Why is the bed not Occupied right after approval?

A: Approval does not mean payment. The bed becomes Occupied only when the move-in fee is verified, so an unpaid applicant cannot hold a bed forever.

8. Move-in payment

What it does

A newly approved tenant can only see the Move-In pages until they pay. They choose full or half payment, pick a payment method (GCash, bank, cash), and upload proof.

How it works

1. The **movein.check** middleware (RedirectPendingMoveInTenant) sends a tenant with status "pending_move_in_payment" back to the Move-In pages whatever page they try to open.
2. Tenant may pay the full amount or **half**. Half reserves the bed for **one month** from the payment date (Payments and Fees Schedule 3.2).
3. When the admin approves a proof that makes the move-in bill fully paid, the system sets the bed to **Occupied**, the tenant to **Active**, and emails a **Move-In Permit**.
4. Every day, the **reservations:expire** job checks half-paid move-in bills. If one month has passed and the rest is still unpaid: the bed goes back to Vacant, the contract is terminated, any waiting proof is rejected, and the account is deactivated.
5. Move-in bills never count as "overdue", so the late-payment process never starts for someone who has not moved in.

Where the code is: `app/Http/Controllers/TenantOnboardingController.php`, `app/Http/Middleware/RedirectPendingMoveInTenant.php`, `PaymentController::activateTenantIfMoveInSettled`, `app/Console/Commands/ExpireMoveInReservations.php`

9. Monthly billing

What it does

Every active contract gets one bill per month. The system checks every day and issues each bill **10 days before its due date** (so the bill for November appears on October 22), and the admin can also click "Generate This Month's Billing".

How it works

1. Take every **active** lease contract.
2. Find the last monthly bill. The new bill starts the day after it ended. For the very first bill, the month the tenant moved in is already covered by the **advance** paid at move-in, so the first bill is for the **next** month.
3. Skip if that month starts more than **10 days from today** (too early to bill), or is after the contract end date. This also prevents duplicate bills.
4. Bill period = the calendar month, cut short if the contract ends mid-month. Mid-month move-in and move-out follow the proration settings (see "Proration" below).
5. Due date = the dorm's rent due day (**the 1st**), from Dormitory Profile settings.
6. Add the utilities and Wi-Fi share unless the dorm marked them as included in rent. Add any unbilled penalties.
7. Save the bill as "unpaid" and send the tenant a notification: "Your bill for November 2026 is ready". Because the bill exists 10 days early, the tenant also sees "Rent is due in X days" reminders before the due date.

Where the code is: `app/Http/Controllers/Api/BillingController.php (generate, generateForContract, issueCutoff, BILL_LEAD_DAYS), routes/console.php`

routes/console.php: billing runs every day automatically

```
Schedule::call(function () {
    app(BillingController::class)->generate(new \Illuminate\Http\Request());
})->daily();
```

In plain words: The daily run is safe to repeat. If a bill for that month already exists, the system just skips that contract, so running it twice never bills anyone twice.

Proration: mid-month move-in and move-out

"Prorating" means charging only for the days a tenant actually stays. The daily price is always the monthly rate divided by the number of days in that month (28, 29, 30 or 31), and amounts are rounded to 2 decimals.

Settings. All of this is controlled by three settings in Dormitory Profile (policies section), so nothing is hard-coded:

Rent due basis: fixed day of the month (Pureza: the 1st, bills cover calendar months) or the tenant's start date (start Oct 16 → due Nov 16, Dec 16...).

Mid-month move-in: "Charged in full" (default, matches Pureza) or "Prorated (unused days credited on the first bill)".

Mid-month move-out: "Prorated (only the days stayed are charged)" (default) or "Charged in full".

Move-in in the middle of a month (calendar-month dorms)

The 1-month advance paid with the move-in fee covers the whole move-in month, so the first monthly bill is for the NEXT month. Example: monthly rate ₱3,000, moves in Oct 16.

Charged in full (default): the tenant keeps the rest of October "for free" as part of the advance. November bill rent = ₱3,000.

Prorated: the unused days of the advance (Oct 1–15) are given back as a credit on the first bill.

```
unused days = 16 − 1 = 15;  credit = 3,000 × 15 ÷ 31 = 1,451.61
November rent = 3,000 − 1,451.61 = 1,548.39
```

The credit can never push rent below ₱0. A tenant who moves in on the 1st gets no credit.

Walk-in contracts (no move-in fee): a contract with no move-in bill is billed from its start date to the end of that month. That first short bill is charged in full, unless move-in proration is on; then it is charged by the day.

Move-out in the middle of a month

If the contract end date falls before the last day of the month, the last bill period stops on the end date. Example: ₱3,000 rate, contract ends Dec 10.

```
Prorated (default): 3,000 × 10 ÷ 31 = 967.74
Charged in full:    3,000
```

Start-date billing

When rent is due on the tenant's start day, every period runs start day to start day (Oct 16 – Nov 15), so a mid-month move-in never needs proration. Only a period cut short by the contract end date is prorated, using the number of days in that period, and the move-out setting still decides prorated vs full.

Things people forget

Only base rent is prorated. The utilities and Wi-Fi share is added in full on every bill (when not included in rent).

Changing a setting does not touch bills already issued. Bills keep the amounts they were created with; the new rule applies from the next generated bill.

Where the code is: app/Http/Controllers/Api/BillingController.php (generateForContract, rentForPeriod, startDatePeriod, endsMidMonthAtContractEnd), app/Models/DormitoryProfile.php (defaults), migration 2026_10_06_000001_add_mid_month_move_out_to_dormitory_profile.php

In plain words: Pureza's rule (full month for move-in) is the default. If a dorm wants fairer billing, the admin flips one dropdown; the code already handles both ways.

Questions the panel might ask

Q: How do you avoid billing a tenant twice?

A: The next period always starts the day after the last bill's end. If that month starts more than 10 days from today, nothing is created yet.

Q: Why is the bill issued 10 days before the due date?

A: So the tenant sees the amount early and gets "due in X days" reminders in the portal. The 10 days is one setting (BILL_LEAD_DAYS, taken from TenantNotificationService::REMINDER_DAYS).

Q: Where do the amounts come from?

A: The monthly rate saved on the contract (room type rate per bed, minus any discount), plus utilities/Wi-Fi shares when not included. Settings live in Dormitory Profile, so the owner can change them without code.

Q: Can the rules be changed for another dorm?

A: Yes. Due day, grace period, penalty %, minimum stay, utilities and proration are settings in Dormitory Profile, not hard-coded.

Q: A tenant moves in on the 20th. Do they pay for a full month?

A: By default yes: the advance rent covers the move-in month. If "Mid-month move-in" is set to Prorated, the 19 unused days are credited on their first monthly bill.

Q: How is the daily rate computed?

A: Monthly rate ÷ days in that specific month, so a day in February costs more than a day in March. Results are rounded to 2 decimals.

Q: What happens if the contract ends on the 10th?

A: By default the last bill charges only 10 days. If "Mid-month move-out" is set to Charged in full, the whole month is billed.

Q: Are utilities prorated too?

A: No. Only rent is prorated; the utility and Wi-Fi share is a fixed monthly amount per bed.

10. Paying bills: proof upload, approval, cash

What it does

Tenants pay outside the system (GCash or bank) and upload a screenshot as proof. An admin checks it and approves or rejects it. Cash paid at the office is recorded directly by the admin. Approved payments get a PDF receipt.

How it works

1. Tenant enters amount, method, payment date and uploads proof (image or PDF, up to 10 MB). Reference number is optional.
2. Checks: the bill is not already paid; the amount is **not more than the remaining balance**; the payment date is not in the future; there is **no other proof already waiting** for this bill.
3. The payment is saved as **pending**. Pending payments do **not** reduce the balance yet.
4. Admin approves: the payment becomes approved, the bill's status is recalculated (partial or paid). If the bill becomes paid, any late-payment follow-up for it is closed right away, so the tenant's portal unlocks immediately.
5. If it was a move-in bill now fully paid, the tenant is activated (Section 8).
6. Admin rejects: a **reason is required** so the tenant knows what to fix.

Remaining balance = bill total minus approved payments only (BillingStatement::remainingBalance).

Where the code is: `app/Http/Controllers/Api/PaymentController.php (submitProof, approveProof, rejectProof, recordCash, receiptPdf), app/Http/Controllers/Api/TenantPortalController.php, app/Services/ReceiptPdf.php`

app/Models/BillingStatement.php

```
public function remainingBalance(): float
{
    $paid = $this->payments()->where('status', 'approved')->sum('amount_paid');
    return max(0, round((float) $this->total_amount - (float) $paid, 2));
}
```

Questions the panel might ask

Q: Why not connect GCash directly?

A: Online payment gateways charge fees and need business verification. The client already uses GCash and bank transfers, so we digitized their current process: upload proof, admin verifies. A gateway can be added later.

Q: How do you prevent fake proofs?

A: Every proof must be approved by an admin who checks it against their GCash/bank account. Only approved payments change the balance.

Q: What if a tenant pays a wrong amount?

A: The system refuses an amount above the remaining balance. Partial payments are allowed; the bill becomes "partial".

11. Grace period and late penalty

What it does

Rent is due on the 1st with a **3-day grace period**. If rent is still unpaid after the 4th, the bill becomes **overdue** and a **one-time 10% penalty of the unpaid rent** is added. It is never added twice and never compounded.

How it works

1. Every day, syncOverdueStatuses looks for unpaid/partial monthly bills whose **due date + grace days** has passed, and marks them overdue.
2. For each overdue bill with no late penalty yet: compute how much **rent** (not utilities or Wi-Fi) is still unpaid **as of the grace deadline**, based on the payment date on the proof.
3. Penalty = unpaid rent × penalty % (10%). Saved with an audit-log entry and added to the bill. The tenant gets a notification.
4. Waits if the tenant has a proof under review that is dated on time.
5. If an admin later approves a proof showing the tenant actually paid on time, the penalty is **waived automatically** with a note why.
6. Admins can also add other penalties and damage charges, waive or reinstate them; every change is recorded in an audit log.

Example: Rent ₱4,200 due Oct 1. Grace until Oct 4. Nothing paid by then → overdue on Oct 5 and penalty = ₱4,200 × 10% = ₱420. New total ₱4,620.

Where the code is: `app/Models/BillingStatement.php (syncOverdueStatuses, graceDeadline, overdueRent, applyLatePenalties, waiveLatePenaltyIfPaidOnTime), app/Http/Controllers/Api/PenaltyController.php, DamageController.php`

The penalty formula

```
$overdueRent = $bill->overdueRent($profile);
$amount = round($overdueRent * $percent / 100, 2);   // e.g. 4200 * 10 / 100 = 420
```

Questions the panel might ask

Q: Why use the payment date on the proof and not the approval date?

A: The Payments and Fees Schedule says the date the tenant actually paid decides if it was on time. A slow admin review should not punish the tenant.

Q: Is the penalty compounded?

A: No. The code only adds a late penalty if the bill has none yet (even a waived one is not re-added).

12. Late-payment follow-up (6-stage delinquency escalation)

What it does

When a bill is overdue, the system follows up automatically in six stages, getting firmer each time. It stops the moment the bill is paid. Admins can pause, reset or clear it for a tenant.

Stage	When (days after grace ends)	What happens
1. Flag	Day 1	Bill marked overdue; admin sees the tenant on the Delinquency page.
2. SMS reminders	Days 2, 4 and 7	Text messages to the tenant; Day 7 is marked URGENT.
3. Portal restriction	Day 14	Tenant can only open Billing and Delinquency pages until paid. SMS sent.
4. Emergency contact	Day 30 (one month unpaid)	SMS to the emergency contact ONLY if they agreed to billing reminders. If not, the tenant gets SMS + email instead.
5. Demand letter	Day 75	A formal PDF demand letter is generated with the amount owed and a deadline (day 89); tenant notified.
6. Blacklist	Day 90 (three months unpaid)	Tenant blacklisted and account deactivated; final SMS. Admin may then issue an eviction notice.

In plain words: Pureza's documents only fix two time limits: the 3-day grace period, and that the owner may terminate once **three months** of rent are unpaid (Payments and Fees Schedule 5.5, Agreement 3.4). The other days were chosen by our team to fit inside that. All four are in one setting, **STAGE_DAYS** in EscalationService.php. For the demo, the Testing Tools move a bill's date back so any stage can be shown right away.

How it works

1. Every day the command **escalation:process** calls EscalationService::processAll.
2. It marks overdue bills (Section 11), then for each overdue bill skips tenants who are already blacklisted or paused by an admin.
3. It counts **days overdue** = days since the grace deadline.
4. It runs every stage the tenant now qualifies for. Each stage first checks the log: if it already **succeeded**, it is skipped (so nobody gets the same text twice).
5. If an SMS fails (no signal, gateway down) the log says "pending" and it is **retried** on the next run.
6. Stage 6 only happens if the earlier stages actually completed.
7. When the bill is paid, resolveSettledEscalations closes the logs and lifts the portal restriction.

The guard that blocks restricted tenants is a middleware: if portal_restricted is true, every page except Billing, Delinquency, notifications and logout redirects to Billing.

Where the code is: `app/Services/EscalationService.php, app/Console/Commands/ProcessEscalations.php, app/Http/Middleware/RestrictDelinquentTenant.php, app/Http/Controllers/DelinquencyController.php, app/Services/TextbeeService.php`

EscalationService::advance (shortened)

```
public const STAGE_DAYS = [3 => 14, 4 => 30, 5 => 75, 6 => 90];
public const STAGE_2_DAYS = [2, 4, 7];

if ($daysOverdue >= $stage3Day) { $this->stage3PortalRestriction($bill); }
if ($daysOverdue >= $stage4Day) { $this->stage4EmergencyContact($bill); }
if ($daysOverdue >= $stage5Day) { $this->stage5DemandLetter($bill); }
if ($daysOverdue >= $stage6Day) { $this->stage6Blacklist($bill); }
```

Admin override

On the Delinquency page an admin can **Pause** (stop for now), **Unpause**, **Reset** (start over at nothing, lift restriction and blacklist) or **Clear** (mark current steps resolved, unlock portal). A reason is always required and saved.

Questions the panel might ask

Q: How are SMS sent?

A: Through Textbee, an SMS gateway app on an Android phone. TextbeeService turns numbers like 0917… into +63917… and sends the text. It returns true/false and never crashes the system if sending fails.

Q: Won't the tenant get the same SMS every day?

A: No. Each step is saved in escalation_logs. A step marked "sent" is never repeated; only failed ones are retried.

Q: What about privacy for the emergency contact?

A: Following Agreement 9.3, the emergency contact is texted only if they ticked consent, and the message contains only the amount, due date and penalty, no other personal details, plus how to opt out.

Q: Does the escalation run by itself?

A: Yes, using Laravel's scheduler (routes/console.php, daily). On the server a cron job runs "php artisan schedule:run" every minute. For the demo we also have Testing Tools to move the clock forward.

13. Maintenance tickets and automatic priority

What it does

Tenants file tickets (repairs, concerns, reports about another tenant). The system automatically sets a priority (**Critical, High, Medium, Low**) and shows the admin **why**, for example: Fire or smoke · "usok".

How it works (rule-based, in English and Filipino)

1. Join the title and description and make it lowercase.
2. Remove safe phrases that contain danger words ("fire drill", "fire exit", "no smoking", "smoke detector") so they don't cause false alarms.
3. **Rule 1 – Danger words, any category:** fire, smoke, sunog, usok, gas leak, nakuryente, collapse, intruder, knife… → **Critical**.
4. **Rule 2 – Words for the chosen category:** e.g. Plumbing: "flood/baha" → Critical, "walang tubig" → High. Electrical: "sparks" → Critical, "brownout" → High.
5. **Rule 3 – Default:** if no words match, use the usual level for that category (e.g. Suggestion → Low).
6. Words are matched as **whole words** only, so "fire" does not match inside "firewall".
7. Admins can change the priority, assign, reply and update status.

Where the code is: `app/Services/TicketPriorityClassifier.php (classify), app/Http/Controllers/TenantTicketController.php, TicketController.php`

Questions the panel might ask

Q: Is this AI?

A: No, it is a rule-based keyword classifier. We chose rules because they are predictable, explainable (it shows the word that triggered it), work offline, cost nothing, and understand Filipino words we listed.

Q: What if it gets the priority wrong?

A: The admin can always change it. The reason shown helps them see why it was chosen.

14. Reviews and automatic moderation

What it does

Tenants who moved out can leave one review. Each review is scanned automatically. If it looks bad, it is **hidden** (never deleted) and the admin sees why. The admin can publish, hide or remove it.

What it checks

- **Bad words** from a list (config/review_moderation.php), including tricks: numbers for letters ("g4go"), stretched letters ("gaaaago"), and spaced-out long words ("t a n g i n a").
- **Spam signs:** links, email addresses, phone numbers (10+ digits), the same character 6+ times, one word repeated over half the comment, mostly CAPITAL letters, keyboard mashing ("sdfghjkl").
- A decision made by an admin is never overwritten by the automatic filter.

Where the code is: `app/Services/ReviewModerationService.php (scan, autoModerate), app/Http/Controllers/ReviewModerationController.php`

15. Notifications, SMS and email

Channel	Used for	Code
In-portal bell	New bill (issued 10 days before the due date), "due in X days" reminders from 10 days before, a reminder during the grace period, overdue notice, late penalty, lease ending (30 days before), deposit refund, restrictions.	app/Models/TenantNotification.php, app/Services/TenantNotificationService.php
Login pop-up	Shown once after each tenant login when bills need attention.	AuthController (show_bill_popup)
SMS (Textbee)	Late-payment reminders and notices (Section 12).	app/Services/TextbeeService.php
Email	Login details, password code, move-in permit, rejection reason, overdue-application alerts to admins, final notices.	app/Mail/

Notifications are created with a unique key, so the same reminder is never created twice for the same bill.

16. Reports and forecasting

What it does

Three reports, each viewable on screen and exportable to **Excel (.xlsx) or PDF** with charts: **Occupancy** (current bed statuses), **Financial** (money collected, penalties, expenses in a date range) and **Forecast** (next 3 months).

How the forecast works (simple averages, no AI)

1. Look back at the last 6 months (only complete months; the current month is left out because it is not finished).
2. **Income per occupied bed** = money actually collected ÷ total occupied bed-months.
3. **Average move-ins** per month from history.
4. For each of the next 3 months: occupied beds = current occupied − leases ending that month + average move-ins (never below 0 or above total beds).
5. Expected income = occupied beds × income per bed, rounded to the nearest ₱100 (an exact figure would look more certain than a guess).
6. Expenses = average of recorded monthly expenses. Net = income − expenses. If no expenses were recorded, net is not shown rather than inventing a number.

Where the code is: `app/Http/Controllers/ReportController.php (computeOccupancy, computeFinancial, computeForecast, export)`

Questions the panel might ask

Q: Is the forecast accurate?

A: It is an estimate based on real history and known lease end dates. It gets better as more months of data are recorded. It shows its assumptions (average move-ins, income per bed) so the owner can judge it.

Q: Why is occupancy only "current" and not historical?

A: Beds keep only their current status. A historical trend is estimated from contract start and end dates instead.

17. Tenant records, move-out and dorm settings

Tenant Manager

Admins view and edit tenant records, deactivate a tenant (move-out), print a **Statement of Account** PDF, and record the **deposit refund**: deposit minus deductions (a note is required when there are deductions; deductions can't exceed the deposit; only one refund per tenant). The tenant is notified.

Where the code is: `app/Http/Controllers/TenantController.php`, `app/Services/StatementOfAccountPdf.php`

Moved-out tenants

A moved-out tenant who has not reviewed yet can still log in, but the **moveout.check** middleware keeps them on the move-out page where they can leave a review.

Dormitory Profile (settings)

Everything specific to the dorm is a setting, not hard-coded: name, photos, logo, permits, amenities, house rules, room types and rates, other charges, payment methods (GCash/bank details and QR), rent due day, grace period, penalty %, minimum stay, whether utilities are included, and the document templates. The real Pureza values are loaded by PurezaStationSeeder.

Where the code is: `app/Http/Controllers/DormitoryProfileController.php`, `app/Models/DormitoryProfile.php`, `database/seeders/PurezaStationSeeder.php`

Activity Log

One timeline of important actions: payments reviewed, applications, contracts, penalties, escalations, review moderation, admin logins. It is built by reading the existing records, so nothing extra has to be remembered to log.

Where the code is: `app/Services/ActivityFeedService.php`

18. Everything that runs automatically every day

Job	What it does	File
Billing	Creates each active contract's bill 10 days before its due date.	routes/console.php → BillingController::generate
escalation:process	Marks overdue bills, adds late penalties, runs the 6-stage follow-up, closes paid ones.	app/Console/Commands/ProcessEscalations.php
applications:notify-overdue	Emails admins about applications waiting 3+ days.	app/Console/Commands/NotifyOverdueApplications.php
reservations:expire	Releases beds of half-paid move-ins after one month.	app/Console/Commands/ExpireMoveInReservations.php

On the server, one cron entry runs **php artisan schedule:run** every minute, and Laravel decides which jobs are due. Any job can also be run by hand, e.g. **php artisan escalation:process**.

19. Security and data protection, in short

- **Passwords and reset codes are hashed**, never stored as plain text.
- **Login lockout** after 5 wrong tries (15 minutes).
- **Role checks on the server** (middleware) for admin, tenant and privilege-only pages.
- **CSRF protection** (Laravel built-in): forms only work when sent from our own pages.
- **Validation** on every form: required fields, file types and sizes, dates, amounts.
- **Database queries go through Laravel's Eloquent**, which protects against SQL injection.
- **Transactions** keep related records consistent.
- **Audit trails**: penalty changes, escalation overrides (with reasons), payment reviewer and time, admin logins.
- **Private documents**: the Agreement and Fees Schedule (which shows the owner's bank accounts) are only shown on the Apply page or to admins; only the Rules are public.
- **Data Privacy Act**: consent required before applying; emergency contact texted only with separate consent.

20. Limitations and honest answers

Panelists respect a team that knows its system's limits. If asked, these are true:

- **Stage days 14, 30 and 75 are our team's choice.** The documents only fix the grace period and the three-month limit (day 90), so the owner can ask us to adjust the others.
- **Payments are verified manually**; there is no direct GCash/bank connection.
- **Ticket priority and review moderation are rule-based**, not AI. Admins can correct both.
- **SMS depends on the Textbee phone** having signal and load. Failed texts are retried automatically.
- **Some admin alerts are only written to the system log** for now (marked as "notification stub" in the code), while tenant-side notifications are real.
- **Occupancy history** is estimated from contracts because beds keep only their current status.
- Open decisions with Pureza: the returning-tenant discount and utilities included or not. Mid-month proration is no longer open code-wise: move-in and move-out are separate settings, so whatever Pureza confirms is one dropdown change.

21. Quick-answer sheet

Questions the panel might ask

Q: What framework did you use and why?

A: Laravel 12 (PHP). It has login, security, database tools, email and a scheduler built in, so we could focus on the dorm's rules. It is free and widely used.

Q: What database?

A: MySQL. Main tables: users, tenants, floors, rooms, beds, applications, lease_contracts, billing_statements, payments, penalties, escalation_logs, maintenance_tickets, reviews.

Q: Where did the stage days come from?

A: Pureza's Payments and Fees Schedule gives a 3-day grace period and allows termination after three months unpaid (5.5). So Stage 6 is at day 90. The rest (day 14 restriction, day 30 emergency contact, day 75 demand letter) are spaced inside that by our team. Fees 5.3 allows limiting portal access while a balance is unpaid.

Q: How does the system know a bill is late?

A: Due date + grace days (1st + 3 days). After that day passes, the daily job marks it overdue (BillingStatement::syncOverdueStatuses).

Q: How is the move-in fee computed?

A: Monthly rate × 2: one month advance + one month security deposit (ApplicationController::approve).

Q: When does a bed become occupied?

A: Only after the move-in fee is fully paid and approved (PaymentController::activateTenantIfMoveInSettled).

Q: How do you generate PDFs?

A: With DomPDF: we design the document as an HTML/Blade page and convert it to PDF. Used for contracts, receipts, statements of account, demand letters, eviction notices and reports.

Q: How is the late penalty computed?

A: 10% (setting) of the rent still unpaid at the end of the grace period, one time only (BillingStatement::applyLatePenalties).

Q: Can a blocked tenant still pay?

A: Yes. A restricted tenant can still open Billing to pay; that is the whole point. Only a blacklisted tenant must settle at the office.

Q: What happens when they pay while restricted?

A: The moment the admin approves the payment and the bill becomes paid, the escalation closes and the portal unlocks (resolveSettledEscalations).

Q: How did you test?

A: Using seeded dummy data (DemoDataSeeder) and the admin Testing Tools that move a record's date forward or run a daily job immediately, so we can show weeks of behavior in minutes.